

Task 9: Advanced Sorting Algorithms & Logic Building

Data Structures & Algorithms — Divide & Conquer Sorting + Array Logic

Task 09

Pichle task mein humne basic sorting (Bubble, Selection, Insertion) dekhi thi. Is task mein level thoda upgrade hota hai — ab hum **Divide and Conquer** approach par challenge, saath hi kuch tricky Java code snippets aur ek optimization challenge bhi solve karenge.

1 Theory: Next-Level Sorting Algorithms

Merge Sort

$O(N \log N)$

Array ko do equal halves mein divide karta hai jab tak single elements nahi bachte, aur fir unko sort karte hue merge karta hai. Time complexity hamesha $O(N \log N)$ rehti hai — best, average aur worst case teeno mein.

→ [GeeksforGeeks: Merge Sort](#)

Quick Sort

$O(N \log N)$ avg

Ek 'pivot' element choose karta hai aur chhote elements ko left aur bade elements ko right mein shift karta hai. Iske baad left aur right sub-arrays ko recursively sort karta hai. Worst case $O(N^2)$, lekin average case mein Merge Sort se bhi fast hota hai in practice.

→ [GeeksforGeeks: Quick Sort](#)

2 DSA Practice Problems

Akshara ke liye

Basic & Implementation Focus — logic building par concentrate karo

1. Subscription Cost CODECHEF

Basic math aur if-else logic ka use karna hai.

Is exact naam se stable public link nahi mil paya — CodeChef ke practice section mein search karo, ya apne course/contest ke problem list se direct link uthao.

2. First and Last Digit CODECHEF

Loop ya string manipulation se number ka pehla aur aakhri digit nikal kar sum karna hai.

→ codechef.com/problems/FLOW004

3. Practice makes us perfect CODECHEF

Array ke elements ko iterate karke ek specific condition (jaise target weeks) check karni hai.

→ codechef.com/problems/PRACTICEPERF

4. Find Second Largest Element GFG

Array ko sort kiye bina, ek single ya double pass mein second largest element dhundna hai.

→ [GeeksforGeeks: Find Second Largest Element](https://www.geeksforgeeks.org/find-second-largest-element/)

5. Reverse an Array GFG

Two-pointer approach use karke array ko in-place reverse karna hai (bina naya array banaye).

→ [GeeksforGeeks: Reverse an Array](https://www.geeksforgeeks.org/reverse-an-array/)

Hardik ke liye

Medium/Hard Logic & Optimization — time complexity matter karegi

1. Merge Intervals LEETCODE 56

Ek 2D array of intervals diya hoga, unhe sort karke overlapping intervals ko merge karna hai.

→ leetcode.com/problems/merge-intervals

2. Kth Largest Element in an Array LEETCODE 215

Bina poora array sort kiye (QuickSelect ya Min-Heap ka use karke) Kth largest element nikalna hai.

→ leetcode.com/problems/kth-largest-element-in-an-array

3. Find the Duplicate Number LEETCODE 287

Array jisme 1 se N tak numbers hain, bina extra space aur array modify kiye duplicate find karna hai (Floyd's Cycle Detection / Tortoise and Hare).

→ leetcode.com/problems/find-the-duplicate-number

4. Count Inversions GFG

Merge Sort ke logic ko modify karke array mein inversions count karne hain (jahan $arr[i] > arr[j]$ and $i < j$).

→ [GeeksforGeeks: Count Inversions of an Array](https://www.geeksforgeeks.org/count-inversions-of-an-array/)

3 Tricky Code Analysis (10 Questions)

In codes ko dry run karo. Output predict karo, ya agar compilation/runtime error hai toh reason batao.

Case 1 — Enhanced For-Loop Mutation

```
public class Test1 {  
    public static void main(String[] args) {  
        int[] arr = {5, 10, 15};  
        for (int x : arr) {  
            x = x * 2;  
        }  
        System.out.println(arr[0]);  
    }  
}
```

Case 2 — Array Initialization & Default Values

```
public class Test2 {  
    public static void main(String[] args) {  
        int[] arr = new int[3];  
        System.out.println(arr[1]);  
    }  
}
```

Case 3 — Array Bounds Trap

```
public class Test3 {  
    public static void main(String[] args) {  
        int[] arr = new int[0];  
        System.out.println(arr.length);  
        System.out.println(arr[0]);  
    }  
}
```

Case 4 — The Length Property vs Method

```
public class Test4 {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3};  
        System.out.println(arr.length());  
    }  
}
```

Case 5 — Array Print Behavior

```
public class Test5 {  
    public static void main(String[] args) {  
        char[] chars = {'a', 'b', 'c'};  
        int[] nums = {1, 2, 3};  
        System.out.println(chars);  
        System.out.println(nums);  
    }  
}
```

Case 6 — Post-Increment in Arrays

```
public class Test6 {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30};  
        int i = 0;  
        System.out.println(arr[i++]);  
        System.out.println(arr[i]);  
    }  
}
```

Case 7 — 2D Array References

```
public class Test7 {  
    public static void main(String[] args) {  
        int[][] matrix = new int[2][];  
        matrix[0] = new int[3];  
        System.out.println(matrix[0][2]);  
        System.out.println(matrix[1][0]);  
    }  
}
```

Case 8 — Out of Bounds on Loop Condition

```
public class Test8 {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3};  
        for (int i = 0; i < 3; i++) {  
            if (arr[i] == 3) {  
                System.out.println(arr[i + 1]);  
            }  
        }  
    }  
}
```

Case 9 — Overwriting References

```
public class Test9 {
    public static void main(String[] args) {
        int[] a = {1, 2};
        int[] b = {3, 4};
        a = b;
        b[0] = 99;
        System.out.println(a[0]);
    }
}
```

Case 10 — Collection Overload

```
import java.util.ArrayList;
public class Test10 {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.remove(10);
        System.out.println(list);
    }
}
```

4 Code Modification Challenge

Neeche ek basic Bubble Sort ka code diya gaya hai jo array ko ascending order mein sort karta hai:

Sorter.java — Starting Point

```
public class Sorter {
    public static void main(String[] args) {
        int[] arr = {5, 2, 9, 1, 5, 6};
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

Tumhara Task — Is code ko modify karo taaki yeh:

1. Array ko **descending** (ghat-te hue) order mein sort kare.

2. Sorting ke dauran total kitne **swaps** hue hain, usko count kare aur aakhiri mein print kare.
3. Agar array pehle se hi sorted hai, toh process ko optimize kare (loop jaldi break ho jaye aur unnecessary iterations na hon — jise *optimized bubble sort* kehte hain).

5 Submission & Deployment

Dono students ko apna code `task_09/` folder mein push karke GitHub link submit karni hai.

```
task_09/
├── Akshara/
│   ├── SubscriptionCost.java
│   ├── FirstAndLastDigit.java
│   ├── PracticeMakesUsPerfect.java
│   ├── SecondLargest.java
│   └── ReverseArray.java
├── Hardik/
│   ├── MergeIntervals.java
│   ├── KthLargestElement.java
│   ├── FindDuplicateNumber.java
│   └── CountInversions.java
├── TrickyCodeAnalysis.md (10 cases ke answers + reasoning)
└── Sorter.java (modified – descending + swap count + optimized)
```